

01 Shoe Pick

Pick up the vintage yellow shoe



CaP-Agent0- Failed

```
# CaP-Agent recomputes perception, point clouds, pos, and quat inline.
point = point_prompt_molmo(rgb, "vintage yellow track shoe").get(
    "vintage yellow track shoe with spikes"
)
mask = segment_sam3_point_prompt(rgb, point)[0]["mask"]
pc_seg = mask_to_world_points(mask, depth, K, T_cam_to_world)
pc_full = mask_to_world_points((depth > 0.1) & (depth < 2.0), depth, K, T_cam_to_world)
try:
    grasps, scores =
        plan_grasp_from_point_clouds(pc_full, pc_seg,
            label="yellow_shoe")
    grasp, _ = select_top_down_grasp(grasps, scores,
        vertical_threshold=0.3)
    if grasp is None:
        grasp = grasps[np.argmax(scores)]
        yaw_90 = np.eye(4)
        yaw_90[:3, :3] = np.array([[0, -1, 0],
            [1, 0, 0], [0, 0, 1]])
        grasp = grasp @ yaw_90
    pos, quat = decompose_transform(grasp)
except Exception:
    obb =
        get_oriented_bounding_box_from_3d_points(pc_seg)
    pos = obb["center"] + np.array([0.0, 0.0, 0.015])
    quat = rotation_matrix_to_quaternion(obb["R"])
# The low-level plan is brittle: every waypoint is hand-authored.

open_gripper()
goto_pose(pos + np.array([0.0, 0.0, 0.12]), quat)
goto_pose(pos, quat)
close_gripper()
goto_pose(pos + np.array([0.0, 0.0, 0.15]), quat)
```

RATS - Success

```
# RATS reuses verified perception instead of rebuilding Molmo/SAM logic.
pc_segment = None
for name in ["track shoe", "brown shoe", "vintage shoe", "shoe"]:
    pc_segment =
        localize_and_verify_object_point_cloud(name,
            camera_name="agentview")
    if pc_segment is not None and
        len(pc_segment) > 0:
        break
if pc_segment is None or len(pc_segment) == 0:
    raise RuntimeError("Failed to localize verified shoe point cloud")
# RATS delegates pos/quat/waypoints to a reusable grasp-and-lift abstraction.
target_pos = np.mean(pc_segment, axis=0)
execute_top_down_grasp_and_lift(
    target_pos,
    grasp_z_offset=0.02,
    pre_grasp_height=0.10,
    lift_height=0.20,
)
```

02 Table Close

Close the table



CaP-Agent0 - Failed

```
# CaP-Agent stays in ad hoc affordance probing instead of calling a close skill.
res_handle = point_prompt_molmo(rgb, "the handle of the table drawer")
res_leaf = point_prompt_molmo(rgb, "the table leaf that is open")
res_drawer = point_prompt_molmo(rgb, "the open drawer")
# The failed path never reaches a reliable pos/quat/action plan.
print("Molmo points:", res_handle, res_leaf, res_drawer)
```

RATS - Success

```
# RATS invokes a learned close-object abstraction directly.
success = push_object_closed(
    "drawer",
    push_distance=0.20,
    approach_distance=0.15,
)
if not success:
    raise RuntimeError("Failed to close drawer")
```

03 Pick and Place

Pick up the spanner and place it in or on the white bowl blue



CaP-Agent0 - Failed

```
# CaP-Agent manually recomputes masks, point clouds, grasp pose, and place pose.
spanner_coord = point_prompt_molmo(rgb, "spanner").get("spanner")
bowl_coord = point_prompt_molmo(rgb, "white bowl blue").get("white bowl blue")
spanner_mask = segment_sam3_point_prompt(rgb, spanner_coord)[0]["mask"]
bowl_mask = segment_sam3_point_prompt(rgb, bowl_coord)[0]["mask"]
spanner_points = mask_to_world_points(spanner_mask, depth, intrinsics, extrinsics)
bowl_points = mask_to_world_points(bowl_mask, depth, intrinsics, extrinsics)
scene_points = mask_to_world_points((depth > 0.1) & (depth < 2.0), depth, intrinsics, extrinsics)
grasp_tfs, grasp_scores =
    plan_grasp_from_point_clouds(scene_points, spanner_points)
grasp, _ = select_top_down_grasp(grasp_tfs, grasp_scores, world_from_grasp_frame=None)
if grasp is None:
    grasp = grasp_tfs[np.argmax(grasp_scores)]
    grasp = grasp @ np.array([[0, -1, 0, 0], [1, 0, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1]])
    pos, quat = decompose_transform(grasp)
    open_gripper()
    goto_pose(pos + np.array([0.0, 0.0, 0.10]), quat)
    goto_pose(pos, quat); close_gripper()
    goto_pose(pos + np.array([0.0, 0.0, 0.15]), quat)
# Placement is also hand-authored from a recomputed bowl centroid.
place_pos = np.mean(bowl_points, axis=0) + np.array([0.0, 0.0, 0.08])
goto_pose(place_pos, quat); open_gripper()
goto_pose(place_pos + np.array([0.0, 0.0, 0.15]), quat)
```

RATS - Success

```
# RATS separates verified localization from reusable manipulation skills.
spanner_pc =
    localize_and_verify_object_point_cloud("spanner")
if spanner_pc is None or spanner_pc.shape[0] == 0:
    raise RuntimeError("Failed to localize verified spanner point cloud")
execute_top_down_grasp_and_lift(
    np.mean(spanner_pc, axis=0),
    grasp_z_offset=0.02,
    pre_grasp_height=0.10,
    lift_height=0.15,
)
bowl_pos = localize_object_with_molmo_sam3("white bowl blue")
if bowl_pos is None or bowl_pos.size == 0:
    raise RuntimeError("Failed to localize bowl")
# RATS delegates transport and release to a reusable grasped-object abstraction.
translation = bowl_pos + np.array([0.0, 0.0, 0.08]) -
    get_observation()["robot_cartesian_pos"][:3]
translate_grasped_object_and_release(
    current_pos=get_observation()["robot_cartesian_pos"][:3],
    current_quat=get_observation()["robot_cartesian_pos"][3:7],
    direction=translation /
        (np.linalg.norm(translation) + 1e-8),
    distance=np.linalg.norm(translation),
    retreat_z=0.15,
)
```

RATS reuses verified perceptual and manipulation abstractions, while CaP-Agent0 direct synthesis repeatedly reconstructs low-level perception, geometry, grasp pose, and waypoint logic inline. In these matched tasks, CaP-Agent0 failed and RATS succeeded.